

Phenoscript Workflow

WHS Methodological Pills

Sergei Tarasov

www.tarasovlab.com



What is Phenoscript?

Phenoscript is a **domain-specific language** for writing machine-readable morphological descriptions using biological ontologies.

Descriptions written in Phenoscript are:

- Based on the **Entity–Quality (EQ)** model
- Linked to terms from standard ontologies (AISM, PATO, UBERON, RO, ...)
- Converted to **OWL knowledge graphs** for reasoning and querying

File formats:

- `.yphs` — default format mixing YAML and Phenoscript (recommended)
- `.phs` — pure Phenoscript

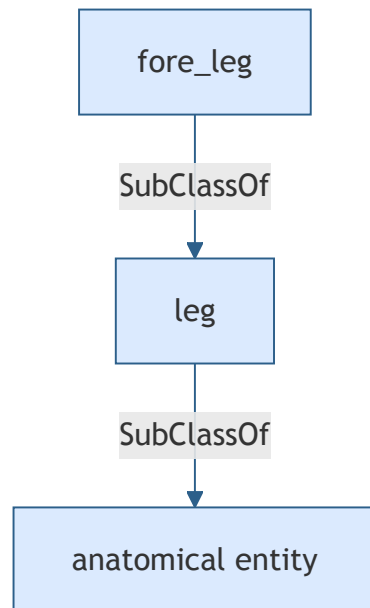
TBox and ABox

OWL knowledge bases have two layers:

TBox — Terminological Box

Defines *classes* and their relationships using ontologies.

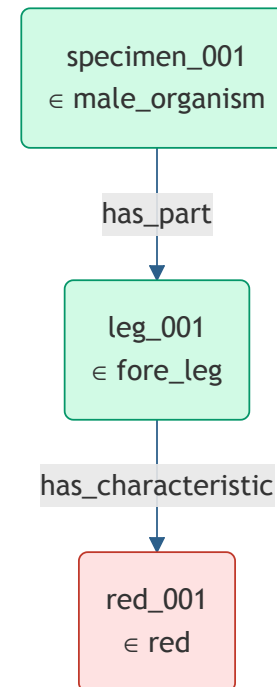
This is the background knowledge.



ABox — Assertion Box

Describes ontology instances (*individuals*) using TBox classes.

Phenoscript generates the ABox from your `.yphs` files.



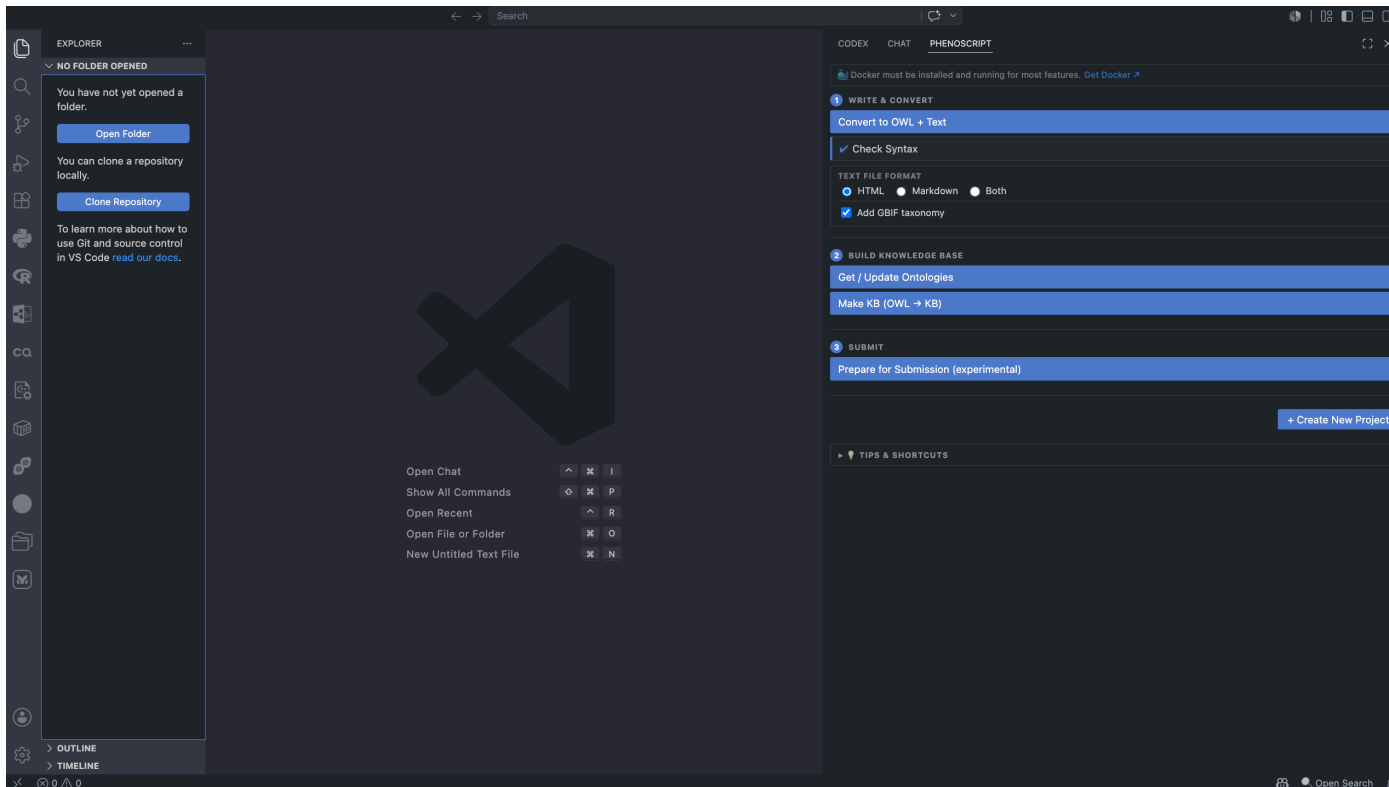
Ontologies Used in Species Descriptions

Abbrev	Full name	Example terms
AIMS	Ontology for the Anatomy of the Insect SkeletoMuscular system	pronotum, wing
COLAO	Coleoptera Anatomy Ontology	elytron, mesoventrite
UBERON	Uberon multi-species anatomy ontology	female organism, adult organism
PATO	Phenotype And Trait Ontology	red, convex, length, setose
BSPO	Biological Spatial Ontology	distal region, ventral side
RO	Relation Ontology	part_of, has_characteristic
BCO	Biological Collections Ontology	catalogNumber, TaxonID
CDAO	Comparative Data Analysis Ontology	TU (taxonomic unit)
IAO	Information Artifact Ontology	denotes
UO	Units of Measurement Ontology	millimeter
TAXRANK	Taxonomic Rank Vocabulary	species
PHS	Phenoscript Ontology	has_trait, OTU Block

For details see obofoundry.org.

Overview — Phenoscript Workflow

1. Create Project and Write your Description
2. Convert to OWL + Natural Text
3. Build Knowledge Base



Step 2 — Convert to OWL + Natural Text

Click **Convert to OWL + Text** in the sidebar.

Converts `.yphs` files to OWL and auto-generates human-readable text, enriched with GBIF taxonomy.

Output	Folder	Notes
OWL files	<code>output/owl_init/</code>	One <code>.owl</code> per <code>.yphs</code> — the semantic descriptions
Natural language	<code>output/nl/</code>	HTML/Markdown auto-generated text
SHACL log	<code>output/log-shacl/</code>	Must contain <code>Conforms</code> — checks graph structure

SHACL validates the structure of your knowledge graph (e.g., that the taxonomy is properly formed). If the log says `Conforms`, all is well.

Step 2: Build Knowledge Base (KB)

Step 2a — Get / Update Ontologies

Click **Get / Update Ontologies** in the sidebar.

- Downloads all ontologies listed in `phs-config.yaml`
- Places them in `source_ontologies/`
- Merges them into a single `tbox.owl` file (the background knowledge)

Do this once per project. Only repeat it if ontologies are updated with new terms during your project.

Step 2: Build Knowledge Base (KB)

Step 2b — Make (OWL → KB)

Click **Make (OWL → KB)** in the sidebar.

This step:

1. Merges all OWL descriptions from `output/owl_init/` with `tbox.owl`
2. Runs the **Whelk reasoner**
3. Produces the final knowledge base: `output/kb/<projectname>-kb.ttl`

If successful, the output panel prints:

```
Success: your dataset is consistent
```

 You must see "**Success: your dataset is consistent**" before proceeding. Inconsistent descriptions cannot be queried reliably.

What the Reasoner Does — 1: Consistency Check

The reasoner verifies that all statements follow the rules defined by the ontologies.

Most common mistake — wrong edge:

```
# Incorrect – quality cannot be a part of a structure
# Meaning: head has_part red color
aism-insect_head > pato-red;

# Correct – quality must be a characteristic of a structure
# head has_characteristic red color
aism-insect_head >> pato-red;
```

`pato-red` is a *quality*. It cannot be a *part* (`>`) of an anatomical entity — it must be a *characteristic* (`>>`). The reasoner will catch this and report an inconsistency.

What the Reasoner Does — 2: Inference

The reasoner also **infers additional relationships** not explicitly stated.

Given:

```
# Meaning: organism has_part head. head has_part anterior_region that is red color
organism > aism-insect_head > bspo-anterior_region >> pato-red;
```

The reasoner automatically infers:

```
# Inferred – because has_part is transitive
# Meaning: organism as_part anterior_region
organism > bspo-anterior_region;

# Inferred – because red is a colour; the described individual also belongs to the class pato-color
pato-red SubClassOf pato-color;
```

This enrichment is what makes traits **automatically queryable across all descriptions**.

Debugging Inconsistencies

The reasoner does not tell you *which* statement caused the error. Two strategies:

Option 1 — Comment Out Code

Select a block of statements and comment/uncomment using:

- **macOS:** `Cmd + /`
- **Windows/Linux:** `Ctrl + /`

Then re-run **Convert** → **Make (OWL → KB)**.

Repeat until you isolate the error.

Option 2 — PhenoScript-GPT

[phenoscript-gpt](#)

Paste the problematic code into **PhenoScript-GPT** and ask it to find the error.

The GPT is trained on Phenoscript syntax and annotation rules and can often pinpoint the problem quickly.